Movies Released After The Year 2000

🗂️ Database Description

The movie_streaming database is designed to manage and monitor digital content consumption across users. It maintains detailed records of user profiles, movie metadata, genre classifications, individual viewing history, and subscription plans. This system enables in-depth analysis of user preferences, viewing trends, and billing activities, supporting both personalized recommendations and subscription management.

📑 Schema Details (Database: movie_streaming)

Column Name	Data Type	Constraints	Description
movie_id	INT	PRIMARY KEY	Unique movie ID
title	VARCHAR(100)	NOT NULL	Movie title
release_year	INT		Year of release
genre_id	INT	FOREIGN KEY → Genres(genre_id)	Genre of the movie

📕 Table: Movies

Catalog of movies available on the platform.

Sample Data

Movies

movie_id	title	release_year	genre_id
901	The Great Escape	1963	2
902	Life of Pi	2012	4
903	The Office (Series)	2005	3
904	Hamlet	1996	1

📌 Problem Statement

The platform team wants to feature all movies released after the year 2000. Your task is to retrieve the list of such movies including their movie_id, title, and release_year, sorted by release_year in ascending order.

📌 Query to be Implemented

Write a query to return all movies released after the year 2000. Output the movie_id, title, and release_year, sorted by release_year.

✅ **Expected Output Format**

movie_id	title	release_year
903	The Office (Series)	2005
902	Life of Pi	2012

M2 Mock 1 : JOIN3_R_2: Products With Total Quantity Sold And Customer Names

🗂️ **Database Description**

The retail_store database is structured to manage and analyze key transactional and operational data within a retail environment. It captures detailed information about customers, product inventory, orders, and item-level purchases. This schema supports querying customer purchase behavior, product sales performance, and order histories, enabling insights into customer trends, product demand, and overall business operations.

📓 **Table: Orders**

Captures order-level metadata including customer and date.

Column Name	Data Type	Constraints	Description
order_id	INT	PRIMARY KEY	Unique order number
customer_id	INT	FOREIGN KEY → Customers(customer_id)	Customer who placed the order
order_date	DATE	NOT NULL	Date the order was placed
total_amount	DECIMAL(10,2)		Total bill amount for the order

Table: Order_Items

Represents items associated with each order, along with quantity and pricing.

Column Name	Data Type	Constraints	Description
order_id	INT	FOREIGN KEY → Orders(order_id)	Order to which item belongs
product_id	INT	FOREIGN KEY → Products(product_id)	Product purchased in the order
quantity	INT	NOT NULL	Number of units ordered
unit_price	DECIMAL(10,2)	NOT NULL	Price per unit at the time of order
PRIMARY KEY	Composite	PRIMARY KEY (order_id, product_id)	Ensures one entry per product/order

✅ Test Data

Customers

customer_id	name	email	phone
201	John Doe	john@example.com	1234567890
202	Jane Smith	jane@example.com	2345678901
203	Emily Davis	emily@example.com	3456789012
204	Mark Wilson	mark@example.com	4567890123

Products

product_id	name	category	price
301	Laptop	Electronics	1000.00
302	Smartphone	Electronics	600.00
303	Office Chair	Furniture	150.00
304	Notebook Set	Stationery	10.00

Orders

order_id	customer_id	order_date	total_amount
401	201	2023-08-01	1010.00
402	202	2023-08-03	1600.00
403	203	2023-08-04	20.00
404	204	2023-08-05	750.00

Order_Items

order_id	product_id	quantity	unit_price
401	301	1	1000.00
401	304	1	10.00
402	301	1	1000.00
402	302	1	600.00
403	304	2	10.00
404	303	5	150.00
404	304	1	10.00
402	303	1	150.00
403	302	1	600.00
401	303	2	150.00
403	301	1	1000.00
404	302	1	600.00

📌 Problem Statement

Management wants a report that shows each product, the **total quantity sold**, and the **names of customers who purchased it**. This will help in analyzing product popularity and customer purchase behavior.

📌 Query to be Implemented

Write a SQL query to retrieve:

- product_name
- total_quantity_sold (sum of all quantities sold)
- customer_name

Each row should represent a product and one of its purchasing customers.

📌 Expected Output

product_name	total_quantity_sold	customer_name
Laptop	3	John Doe
Laptop	3	Jane Smith
Laptop	3	Emily Davis
Notebook Set	4	John Doe
Notebook Set	4	Emily Davis
Notebook Set	4	Mark Wilson
Office Chair	8	John Doe
Office Chair	8	Jane Smith
Office Chair	8	Mark Wilson
Smartphone	3	Jane Smith
Smartphone	3	Emily Davis
Smartphone	3	Mark Wilson

Sample Data

Doctors

doctor_id	name	specialization	department_id
504	Dr. Nair	Orthopedic Surgeon	4
502	Dr. Rao	Neurologist	2

Appointments

appointment_id	doctor_id	appointment_date	status
704	504	2023-08-05	Scheduled
708	501	2023-08-09	Scheduled
712	502	2023-08-13	Completed

📌 Problem Statement

The scheduling desk wants to know which doctors have appointments that are currently in "Scheduled" status. For each appointment, display doctor name, specialization, and the scheduled date. Filter only appointments with status = 'Scheduled'.

📌 Query to be Implemented

Write a query to return appointment_id, appointment_date, doctor_name, and specialization for scheduled appointments.

📌Expected Output

appointment_id	appointment_date	doctor_name	specialization
704	2023-08-05T00:00:00.000Z	Dr. Nair	Orthopedic Surgeon
708	2023-08-09T00:00:00.000Z	Dr. Mehta	Cardiologist

Monthly Revenue Change

Concepts Tested

LAG() across ordered months

Arithmetic difference from previous month

Date-based ordering (ORDER BY month_start)

Problem Statement

Given a monthly revenue summary, show **each month's revenue** along with the **change vs the previous month** (current minus previous).

Required columns: month_start, total_revenue, prev_month_revenue, revenue_change.

Schema Details (Database: SALES_SUMMARY_DB)

MonthlySales Table

Column Name	Data Type	Description
month_start	DATE (PK)	First day of the month (e.g., 2025-11-01)
total_revenue	DECIMAL(12,2)	Total revenue for that month

Query to be Implemented

Write a single query to:

• Order rows by month_start

• Use LAG(total_revenue) to fetch the previous month’s revenue

• Compute revenue_change = total_revenue - prev_month_revenue

• Output the required columns

Expected Output Format (Exact)

month_start	total_revenue	prev_month_revenue	revenue_change
2025-11-01T00:00:00.000Z	95000.00	90000.00	5000.00
2025-10-01T00:00:00.000Z	90000.00	87000.00	3000.00
2025-09-01T00:00:00.000Z	87000.00		